

TESTOWANIE OPROGRAMOWANIA

ANDRZEJ DANILUK

Przypadek testowy - zbiór danych wejściowych, wstępnych warunków wykonania, oczekiwanych rezultatów i końcowych warunków wykonania opracowany w określonym celu lub dla warunku testowego, jak wykonanie pewnej ścieżki programu lub zweryfikowanie zgodności z konkretnym wymaganiem

Elementy PT:

ID - unikalny identyfikator

Warunki wstępne uruchomienia

Kroki do wykonania testu

Oczekiwany rezultat

Warunek końcowy

RODZAJE TESTÓW

Testy można rozróżniać z uwzględnieniem podziału na fazę lub rodzaj. W kolejności wykonywania danego rodzaju testów w procesie wytwarzania oprogramowania są to (ze względu na fazę):

1. testy jednostkowe,
2. integracyjne,
3. funkcjonalne,
4. systemowe,
5. akceptacyjne.

IEEE (ang. *Institute of Electrical and Electronics Engineers*) w dokumencie IEEE 610.12 z roku 1990 zdefiniowała pojęcia pięciu rodzajów testów.

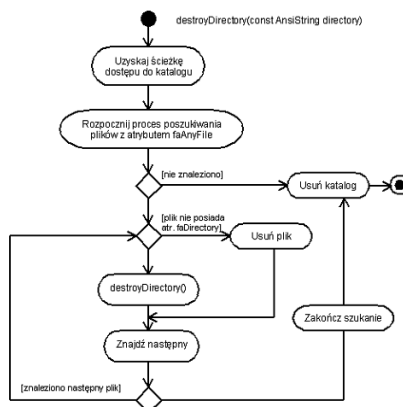
1. Testy jednostkowe (nazywane również modułowymi; ang. *unit tests*) są bezpośrednio związane z pisanym kodem – testują jego małe, niepodzielne fragmenty. O określeniu czym są w danym projekcie te moduły decyduje często sam programista, ponieważ to do jego obowiązków należy wykonywanie testów jednostkowych. W większości przypadków są to funkcje lub metody w klasach, rzadziej całe klasy.

Do każdej z testowanych funkcji pisze się kilka przypadków testowych z uwzględnieniem wiedzy o testowanym kodzie (tzw. testy białej skrzynki). Do funkcji przekazuje się parametry testowe (**warunki wstępne**), po czym bada się zwracane wartości (**warunki końcowe**) i sprawdza ewentualnie generowane wyjątki. W funkcjach sprawdza się wszystkie istotne ścieżki przepływu czynności.

- każda możliwa ścieżka w każdej funkcji została przetestowana
- jeżeli ścieżki są niemożliwe do sprawdzenia z powodu istnienia pętli, to:

W celu zapobieżenia niemożności sprawdzenia kodu z powodu istnienia pętli stosuje się dwie grupy testów:

- Boundary test – działania w pętli nie są wykonywane lub działania w każdej pętli są wykonywane raz dodatkowo wszystkie ścieżki wewnątrz pętli są raz wykonane.
- Interior test – działania we wnętrzu pętli uważa się za przetestowane, jeśli zostały wykonane wszystkie ścieżki, które są możliwe przy dwukrotnym powtórzeniu pętli.

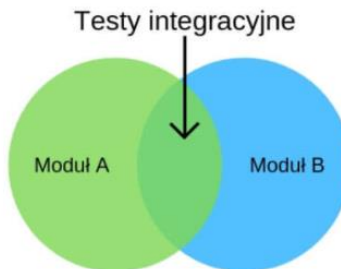


W przypadku *klas*, tworzy się obiekty z zadany stanem początkowym (**czym jest stan obiektu?**), a po serii testowych przekształceń sprawdza się ich stan końcowy. Testy jednostkowe można także zastosować w testowaniu komponentów (**czym jest komponent?**).



Schemat testów jednostkowych

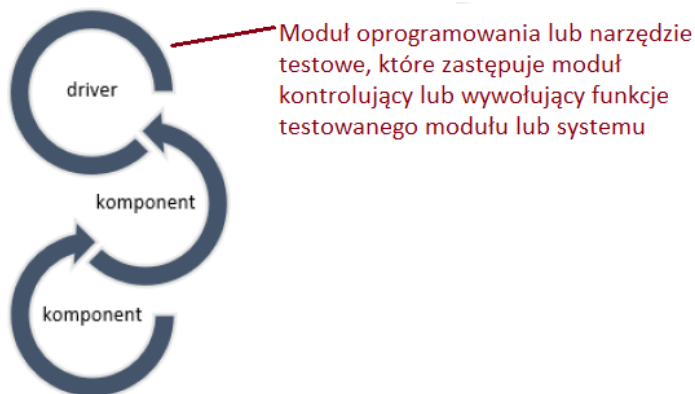
2. Testy integracyjne (ang. *integration tests*) sprawdzają czy kilka modułów współgra ze sobą w kodzie w prawidłowy sposób i czy interakcje pomiędzy integrowanymi modułami (czasem systemami) nie skutkują nieprzewidzianymi błędami. Testowane są przeważnie interfejsy lub konfiguracja systemu.



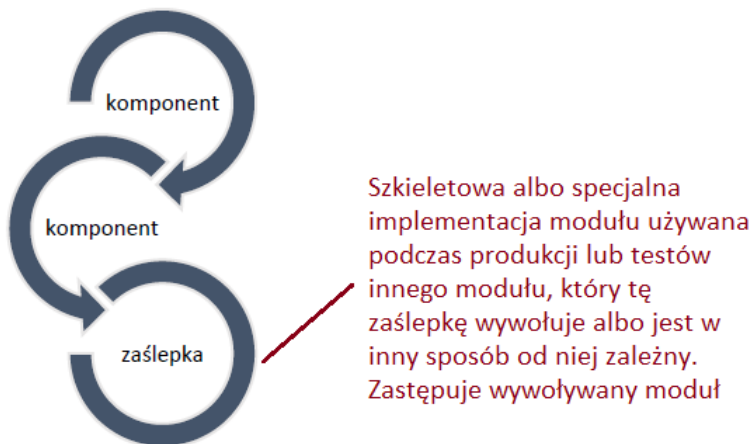
Wyróżnia się kilka sposobów przeprowadzania testów integracyjnych:

- **model wielkiego wybuchu,**
- **integrację z dołu,**
- **integrację z góry.**

W modelu wielkiego wybuchu (ang. *big-bang testing*) testowane są wszystkie integrowane moduły na raz – konstruowany jest mały system stworzony z modułów i sprawdza się jego działanie.



W **integracji z dołu** (ang. *bottom-up testing*) testowane są po kolei komponenty od położonych najniżej w hierarchii projektu (tzn. podstawowych, które nie wymagają innych komponentów do działania), do tych położonych wyżej. W tym modelu używa się tzw. driverów do symulowania komponentów położonych wyżej w hierarchii.



Integracja z góry (ang. *top-down testing*) zaczyna testowanie od elementów najwyżej położonych koncepcyjnie i postępuje w dół hierarchii. Tak zwane zaślepki symulują najniżej położone komponenty. Ze względu na swoją charakterystykę, model wielkiego wybuchu nie nadaje się do początkowych testów – w przypadku wykrycia defektów trudno je zlokalizować, a także łatwo przeoczyć jakiś przypadek testowy.

Po uzyskaniu pewności, że wszystkie moduły składające się na dane funkcjonalności ze sobą dobrze współdziałają, można rozpocząć:

3. Testy funkcjonalne (ang. *functional tests*). Mają na celu ocenę spełnienia wymagań określonych w trakcie projektowania systemu i poprawności działania poszczególnych funkcjonalności, jeszcze zanim staną się częścią całego systemu. Są one wykonywane w stylu **testów czarnej skrzynki**, - **sprawdzają wyłącznie poprawność wyniku działania danej funkcjonalności; nie jest brany pod uwagę wewnętrzny mechanizm odpowiadający za daną funkcjonalność**. Dla funkcjonalności działających jak maszyna stanów buduje się w dokumentacji tablice decyzyjne i na ich podstawie przeprowadza testy. Dla reszty określa się klasy równoważności (zbiory danych, których użycie skutkuje takim samym rezultatem) i wartości brzegowe.



4. Testy systemowe (ang. *system tests*) przeprowadzane są już na kompletnym, w pełni zintegrowanym systemie i sprawdzają zgodność systemu z założeniami. Na tym etapie testuje się powstały system w rzeczywistym środowisku, bez symulatorów.

5. Testy akceptacyjne (ang. *acceptance tests*) to ostatni etap testów, często wykonywany przez klienta, który sprawdza czy produkt końcowy spełnia oczekiwania i wymagania. Na tym etapie wykonywane są scenariusze symulujące wykorzystanie systemu przez użytkownika końcowego. Rozróżniamy wśród nich w szczególności testy alfa i beta (ang. *alpha testing, beta testing*), gdzie w

- **testach alfa** scenariusze są wykonywane przez testerów, którzy są częścią zespołu tworzącego oprogramowanie, natomiast w fazie
- **testów beta** niezupełnie gotowy produkt jest oddawany w ręce prawdziwych użytkowników, którzy mogą zgłaszać błędy twórcom.

Ze względu na rodzaj można wyróżnić:

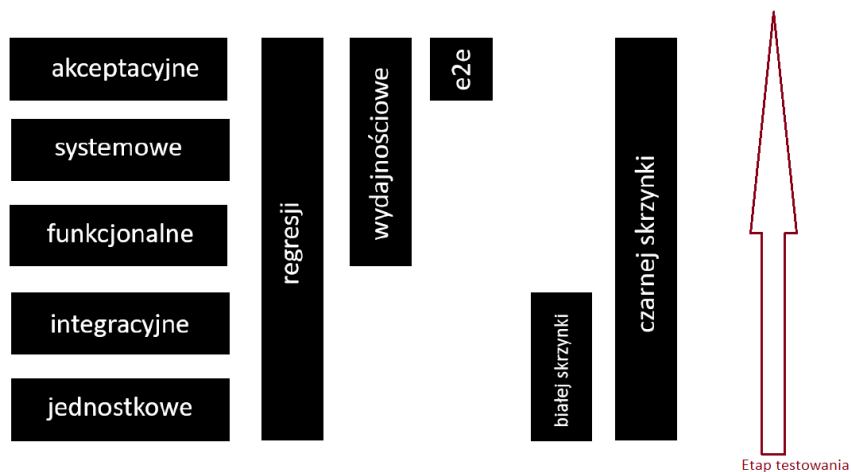
- **testy regresji** (ang. *regression testing*) – weryfikacja, czy po nowowprowadzonych do systemu zmianach (**refactoring ?**), inne jego elementy (do tej pory działające) nie uległy awarii. Skutkiem tych zmian może być błędne działanie innej funkcji programu, która w poprzednich wersjach działała prawidłowo. Wykonywanie testów regresji związane jest z ponownym uruchomieniem zestawu testów, które wcześniej kończyły się poprawnie.
- **testy wydajnościowe** (ang. *performance testing*) – mierzenie wydajności budowanego systemu w różnych kontekstach, obserwuje się między innymi pamięć, procesor, statystyki połączeń,
- **testy statystyczne** – badanie niezawodności systemu na podstawie miar takich, jak: prawdopodobieństwo wystąpienia błędu, częstotliwość występowanie błędu, czas dostępności systemu,
- **testy bezpieczeństwa** (ang. *penetration testing*) – zwane także testami penetracyjnymi, to symulowane ataki na testowany system,
- **testy białej skrzynki** (ang. *white-box testing*) – sposób testowania, w którym uwzględnia się znajomość wewnętrznego działania komponentów systemu (czasem oznacza to także znajomość kodu),
- **testy czarnej skrzynki** (ang. *black-box testing*) – sposób testowania, w którym nie uwzględnia się wiedzy na temat wewnętrznego działania komponentów systemu, a bierze pod uwagę jedynie wynik testowania i stan lub działania początkowe,
- **testy end-to-end** (ang. *end-to-end testing*) – wykonywanie pełnych scenariuszy przypadków użycia od początku do końca,
- **testy dostępności** (ang. *accessibility testing*) – testy dostępności treści dla osób niepełnosprawnych (przeprowadzane m.in. na aplikacjach webowych),
- **testy eksploracyjne** – nieformalne testy oparte na doświadczeniu

testera, niezwiązane z dokumentacją dotyczącą planowanych testów,

- **testy dymu** (ang. *smoke tests*) – typ testów regresji, polegający na wstępnych, powierzchownych testach ujawniających niepowodzenia na tyle kluczowe, aby odrzucić testowaną wersję oprogramowania,

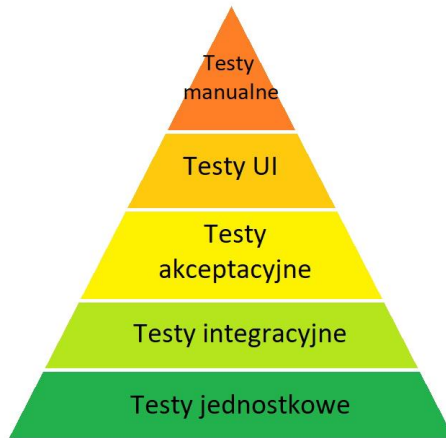
- **testy poczytalności** (ang. *sanity tests*) – typ testów regresji, które są bardziej rozbudowane od testów dymu i sprawdzają szerszy zakres funkcjonalności, ale wciąż są płytsze niż typowe testy regresji – mają za zadanie sprawdzić czy warto poświęcać zasoby na dogłębne testowanie (np. przed wyborem wersji kandydującej do wydania),

- **fuzz testing** – technika, w której dostarcza się nieprawidłowe lub losowe dane wejściowe i monitoruje pojawianie błędów, testy są zautomatyzowane,



Relacje faz i typów testów

- testy białej skrzynki przeprowadza się w początkowych etapach testowania,
- czarnej skrzynki można wykonywać na dowolnym etapie
- regresji można wykonywać na każdym etapie testowania.
- wydajnościowe lub *end-to-end* na choćby częściowo kompletnych systemach - na rysunku wykluczone są z faz testowania jednostkowego i integracyjnego, a testy *end-to-end* są wykluczone dodatkowo z faz testowania systemowego oraz funkcjonalnego, ponieważ zwyczajowo przeprowadza się je na samym końcu, na takiej postaci systemu, która zostanie przekazana użytkownikowi.



Piramida testów

Testowanie automatyczne

Testowanie programów polega na wielokrotnym powtarzaniu tej samej sekwencji działań. Człowiek wykonujący takie testy nie tylko nie wykorzystuje w pełni swojego potencjału pracowniczego, ale może wręcz bardzo szybko zacząć wykonywać powierzone mu zadanie w sposób nieodpowiedni – błędy wynikające ze znużenia.

Automatyzacja jest odpowiedzią na problem zwiększenia efektywności tej fazy wytwarzania oprogramowania. Warto zwrócić uwagę na to, że automatyzacja jest możliwa między innymi dlatego, że działania testera są w wielu przypadkach możliwe do zastąpienia lub symulowania przez maszynę.

Zaletą automatyzacji jest także fakt, że komputer nie generuje błędów wynikających ze znużenia i dlatego wielokrotne powtarzanie tej samej sekwencji zostanie zawsze wykonane przez komputer bezbłędnie i bez pominięcia jakichkolwiek kroków. Dzięki obu tym cechom charakterystycznym dla automatyzacji, faza testów ma największy potencjał jeśli chodzi o redukcję kosztów projektu, choć to także nie jest regułą. Jednak zanim o tym, należałoby odpowiedzieć na pytanie czym konkretnie jest testowanie automatyczne.

Testy automatyczne to testy oprogramowania zaprojektowane i wdrożone w taki sposób, aby powtarzalne procedury testujące były wykonywane bez udziału testera. Takie testy mają najczęściej postać kodu, a konkretniej funkcji lub skryptów napisanych w dedykowanych językach lub frameworkach. Są projektowane, implementowane i uruchamiane przez testera.

Po zakończeniu testu, wyniki lub logi także są sprawdzane przez człowieka, jednak sam przebieg testu odbywa się automatycznie.

Zazwyczaj testy automatyczne są rodzajem oprogramowania, czasem jednak za automatyzację testów odpowiada dedykowany sprzęt.

Cechy testowanego systemu oraz kryteria wpływające na automatyzację

Sens i sukces automatyzacji procesu testowania zależy w dużym stopniu od samego systemu, które będzie mu poddany (ang. *SUT* – *System Under Test*, system poddany testowaniu). Na przykład, łatwiej zautomatyzować jest testowanie systemu, na który składa się kilka prostych plików z kodem napisanym w jednym z powszechnie używanych języków programowania, a trudniej działanie wszystkich połączeń w skomplikowanej sieci Ethernet. Da się z góry określić pewne czynniki wpływające na to, czy automatyzacja będzie łatwiejsza do przeprowadzenia, czy też będzie wymagała więcej wysiłku; są to czynniki wpływające poniekąd na ocenę sensowności automatyzacji.

ISTQB (ang. *International Software Testing Qualifications Board*, międzynarodowa organizacja non-profit zajmująca się standaryzacją testowania oraz certyfikowaniem kompetencji testerów) rozróżnia:

- Czynniki wpływające na automatyzację testów (determinują przede wszystkim narzędzia jakie należy zastosować),

- **interfejsy systemu** – często determinują wybór sposobu komunikacji między skryptami testowymi, a systemem, ale ich niezawodność także wymaga przetestowania,
- **dodatkowe oprogramowanie** używane przez system (ang. *third-*

party software) – każdy element wykorzystywany przez oprogramowanie może wpływać na wynik jego działania,

- **poziom ingerencji** (ang. *level of intrusion*) – mówi o tym, ile modyfikacjom trzeba poddać system, aby umożliwić testowanie (np. czy konieczne jest stworzenie nowego interfejsu). Im niższy poziom ingerencji, tym lepiej, ponieważ testowany system będzie w postaci jak najbardziej zbliżonej do tej, która będzie używana przez użytkownika końcowego,
- **rozmiar i złożoność systemu** (ang. *size and complexity of the SUT*) – będą wpływały na rozmiar i złożoność testów.

- Kryteria automatyzacji (rozstrzygnięcie czy automatyzacja będzie mieć w ogóle sens w danym projekcie):

- **częstotliwość użycia** (ang. *frequency of use*) – im częściej się testuje, tym automatyzacja jest bardziej opłacalna,
- **złożoność testów i systemu** (ang. *complexity to automate*) – im bardziej skomplikowany, tym bardziej projekt zyska na automatyzacji długoterminowo (szczególnie przy częstym użyciu),
- **kontrolowalność systemu** poddawanego testom – powinien być kontrolowalny w dużym stopniu, tzn. działanie systemu powinno być maksymalnie deterministyczne, co nie zawsze jest oczywiste w przypadku skomplikowanych systemów z wieloma zależnościami.

Testy automatyczne można zaklasyfikować jako rodzaj dynamicznej weryfikacji systemu. Do walidacji zazwyczaj niezbędny jest czynnik ludzki, który pozwoli określić czy oprogramowanie faktycznie spełnia wymagania użytkownika końcowego, podobnie do jednej z form statycznej inspekcji kodu, jaką jest czytanie kodu w celu znalezienia błędów.

Test półautomatyczny - jest połączeniem dwóch poprzednich. Tester używa narzędzi pomocniczych do testowania aplikacji. Ten rodzaj ułatwia

testowanie np. interfejsu użytkownika (testy UI).

Wybrane narzędzia

1. Selenium - uznany przez organizację W3C (World Wide Web Consortium) Standard, który przyczynia się do popularyzacji automatyzacji testów funkcjonalnych.

2. HP Unified Functional Testing - wspiera testy funkcjonalne i regresyjne na najbardziej popularnych systemach operacyjnych i przeglądarkach internetowych. Umożliwia również testowanie API i web serwisy.

3. Ranorex - framework do automatyzacji testów GUI.

4. TestComplete SmartBear - platforma automatycznych testów funkcjonalnych.

5. Robot Framework - generyczna platforma do automatyzacji testowania dla testów akceptacyjnych i testów opartych na testach akceptacyjnych (ATDD).

6. Sahi - narzędzie do automatyzacji i testowania aplikacji internetowych dostępne w wersji open-source i komercyjnej.

7. Watir - biblioteka Ruby o otwartym kodzie źródłowym do automatyzacji testów w przeglądarkach internetowych. Obsługuje przeglądarki Internet Explorer, Firefox, Chrome, Opera i Safari. Watir został opracowany przez Breta Pettichorda i Paula Rogersa.

8. Telerik Test Studio - narzędzie do testowania oprogramowania, który służy do automatyzacji testów funkcjonalności stron internetowych i aplikacji desktopowych oraz testowania aplikacji mobilnych. Narzędzie jest dostarczane z wtyczką do programu Visual Studio i samodzielną aplikacją, która korzysta z tych samych repozytoriów i formatów plików. Może również służyć do testów wydajnościowych.